

Архитектура Single Page Applications (SPA)

1. Эволюция веба: от MPA к клиентскому рендерингу

Классическая модель Multi-Page Application (MPA) опирается на серверную генерацию каждого отдельного документа. При любом переходе по ссылке или отправке формы браузер полностью сбрасывает текущий контекст страницы и инициирует тяжелый цикл обработки:

```
[Пользователь] → Клик по ссылке → [Браузер] → GET /catalog.html  
→ [Веб-сервер / Backend Engine] ← [Браузер] ← Полный  
HTML-документ (200 OK) → [Браузер] → Уничтожение старого DOM →  
Парсинг HTML → Загрузка CSS/JS → Вычисление Layout (Reflow) →  
Отрисовка пикселей (Repaint)
```

В этой схеме веб-сервер выполняет двойную работу: запрашивает данные из базы и склеивает их с HTML-разметкой. Браузер при каждом переходе тратит системные ресурсы на повторную компиляцию стилей и скриптов, порождая «эффект белого экрана» (мерцание).

Концепция **Single Page Application (SPA)** кардинально меняет вектор ответственности, перенося генерацию разметки на сторону клиента (Client-Side Rendering, CSR). Сервер единожды отдает минимальный пустой HTML-каркас и скомпилированный JavaScript-бандл:

```
<!DOCTYPE html>  
<html lang="ru">  
  <head>  
    <meta charset="UTF-8">  
    <link rel="stylesheet" href="/assets/bundle.css">  
  </head>  
  <body>  
    <div id="root"></div>  
    <script src="/assets/bundle.js"></script>  
  </body>  
</html>
```

После первичной загрузки браузер больше не запрашивает HTML-страницы. Все дальнейшие экраны и списки рендерятся локально клиентским скриптом, который обменивается с сервером исключительно структурированными данными в формате JSON:

2. Механика клиентской маршрутизации (Client-Side Routing)

Главная инженерная задача SPA — предоставить пользователю привычный интерфейс с уникальными URL, закладками и поддержкой кнопок «Назад»/«Вперед» без перезагрузки окна.

Историческая хэш-маршрутизация (`/#/profile`) уступила место стандарту **HTML5 History API**:

- `history.pushState(state, title, url)` — добавляет запись в историю браузера и меняет отображаемый адрес без отправки HTTP-запроса на сервер.
- `history.replaceState(state, title, url)` — модифицирует текущую запись в стеке истории.
- `window.addEventListener('popstate', callback)` — перехватывает клики по навигационным кнопкам браузера.

Упрощенный принцип работы легковесного SPA-маршрутизатора:

```
class ClientRouter {
  constructor(routes) {
    this.routes = routes;
    this.root = document.getElementById('root');

    window.addEventListener('popstate', () => this.render(window.location.pathname));

    document.body.addEventListener('click', (e) => {
      const target = e.target.closest('a');
      if (target && target.getAttribute('href').startsWith('/')) {
        e.preventDefault();
        const path = target.getAttribute('href');
        window.history.pushState({}, '', path);
        this.render(path);
      }
    });
  }

  render(path) {
    const component = this.routes[path] || this.routes['404'];
    this.root.innerHTML = component();
  }
}
```

⚠ Требование к конфигурации веб-сервера

Поскольку физически на сервере существует только один файл `index.html`, любой входящий GET-запрос по произвольному пути (например, `/profile/billing`) должен перенаправляться веб-сервером (Nginx, Caddy, Vercel) на базовый `index.html` с кодом 200 OK. Иначе браузер получит ошибку 404.

3. Инженерные компромиссы и метрики SPA

При переходе на архитектуру SPA инженеры сталкиваются с рядом компромиссов:

- **Размер JS-бандла (Bundle Size):** При разрастании кодовой базы первичная загрузка приложения по медленной сети замедляется.
- **Индексация поисковыми роботами (SEO):** Хотя современные краулеры исполняют JS, время постановки страниц в очередь рендеринга кратно превышает обход готового серверного HTML.
- **Утечки оперативной памяти:** Страница в SPA живет часами. Забытые слушатели событий и не отмененные подписки приводят к прогрессирующей деградации производительности вкладки.

Декларативная парадигма и JSX

1. Декларативный подход: $UI = f(state)$

В императивной модели разработчик вручную пишет последовательность инструкций по изменению дерева документов. При росте приложения это приводит к потере синхронизации между переменными в коде и реальным DOM.

React утверждает декларативную формулу: пользовательский интерфейс является строгой проекцией текущего состояния приложения:

$$UI = f(state)$$

Разработчик декларирует: «интерфейс должен выглядеть так при данных X». Всю работу по вычислению разницы и обновлению экрана библиотека берет на себя.

2. Компонентная модель и декомпозиция

Компонент — это независимый модуль, инкапсулирующий разметку, локальные стили и бизнес-логику. В React компонент является обычной функцией, обязанной соблюдать математическую чистоту (purity) относительно своих входных параметров: она не должна мутировать глобальные переменные во время рендеринга.

3. Механика JSX и фаза компиляции

JSX — расширение синтаксиса JavaScript, позволяющее описывать структуру дерева компонентов в HTML-подобной форме. Браузеры не понимают JSX нативно; сборщики (Babel, SWC) транпилируют его в вызовы функций ядра:

```
// Исходный JSX:
const element = <h1 className="hero">Архитектура React</h1>;

// Результат компиляции (современный jsx-runtime):
import { jsx as _jsx } from 'react/jsx-runtime';
const element = _jsx('h1', { className: 'hero', children: 'Архитектура React' });
```

Функция `_jsx` возвращает обычный JavaScript-объект — так называемый **React Element**:

```
{
  $$typeof: Symbol.for('react.element'), // Защита от XSS-инъекций
  type: 'h1',
  key: null,
  ref: null,
  props: { className: 'hero', children: 'Архитектура React' }
}
```

4. Однонаправленный поток данных (Props & State)

Данные в React передаются строго сверху вниз — от родительских компонентов к дочерним через неизменяемый (immutable) объект `props`. Дочерний компонент не имеет права менять входные пропсы. Если требуется изменить данные, дочерний узел вызывает переданный родителем колбэк (паттерн «Поднятие состояния» / Lifting State Up).

Virtual DOM и архитектура Fiber

1. Цена прямых манипуляций с реальным DOM

Реальный DOM браузера — тяжеловесный объект C++ с сотнями внутренних свойств. Прямое обращение к нему сопряжено с затратами на цепочку графических вычислений движка рендеринга:

```
[Мутация DOM узла] → Recalculate Style → Layout (Reflow) → Paint (Repaint) → Composite (GPU)
```

Неконтролируемые чтения и записи геометрии (например, чтение `offsetHeight` сразу после изменения ширины) вызывают **Layout Thrashing** — принудительный сброс расчетов браузера, блокирующий поток на десятки миллисекунд.

2. Концепция Virtual DOM

Virtual DOM — легковесное дерево JavaScript-объектов в оперативной памяти. Изменение тысяч узлов в виртуальном дереве происходит за доли миллисекунды, так как оно полностью изолировано от графического конвейера браузера.

3. Эвристический алгоритм согласования (Diffing)

Теоретическая сложность сравнения двух деревьев составляет $O(n^3)$. Для дерева из 1000 элементов это потребовало бы миллиарда операций на каждый кадр. React использует эвристический алгоритм сложности $O(n)$, опирающийся на два постулата:

1. **Элементы с разными типами создают разные деревья.** При смене `<div>` на `<section>` React полностью уничтожает старое поддерево вместе со всем состоянием.
2. **Стабильность элементов списка определяется ключом `key`.**

```
// ❌ Антипаттерн: использование индекса массива при динамическом списке
{items.map((item, index) => <Card key={index} data={item} />)}

// ✅ Корректно: стабильный уникальный идентификатор записи
{items.map((item) => <Card key={item.id} data={item} />)}
```

4. Движок React Fiber

Начиная с версии 16, ядро React построено на архитектуре **Fiber**. Fiber преобразовал рекурсивный синхронный рендеринг в итеративный обход двусвязного списка узлов.

Фаза рендера (Render Phase)	Фаза фиксации (Commit Phase)
Асинхронная, прерываемая браузером.	Синхронная, непрерывная.
Обход Fiber-дерева, вызов компонентов, расчет diff.	Мутация реального DOM, вызов layout-эффектов.
Не производит изменений, видимых пользователю.	Отрисовывает финальные пиксели на экране.

Благодаря Fiber тяжелые вычисления делятся на кванты времени по 5–10 мс (Time Slicing). Если во время фонового рендера пользователь нажимает клавишу, React прерывает работу, отдает главный поток браузеру для плавной реакции ввода, а затем возобновляет построение интерфейса.

Система хуков и управление состоянием

1. Внутренняя структура хуков

Внутри Fiber-узла данные хуков хранятся в виде односвязного списка в поле `memoizedState`. Это объясняет, почему хуки запрещено вызывать внутри условий, циклов и вложенных функций: порядок вызовов хуков обязан строго совпадать между рендерами для корректного чтения ячеек памяти.

2. Хук `useState` и автоматический батчинг

Вызов сеттера состояния планирует новый рендер компонента. В современных версиях React работает **Automatic Batching**: несколько последовательных обновлений состояния внутри таймеров, промисов и обработчиков объединяются в один рендер:

```
function handleAction() {  
  // Вызов ровно ОДИН повторный рендер, а не два  
  setCount(c => c + 1);  
  setFlag(f => !f);  
}
```

3. Жизненный цикл и сайд-эффекты: `useEffect` vs `useLayoutEffect`

- `useEffect` : Выполняется асинхронно *после* отрисовки кадра (Paint). Используется для сетевых запросов, подписок и таймеров, не блокируя вывод интерфейса.
- `useLayoutEffect` : Выполняется синхронно *до* отрисовки кадра, сразу после мутации DOM. Применяется для синхронных замеров геометрии элементов (тултипы, поповеры) во избежание визуального мерцания.

```
useEffect(() => {
  const handler = () => console.log('Скролл');
  window.addEventListener('scroll', handler);

  // Функция очистки (Cleanup) для предотвращения утечек памяти
  return () => window.removeEventListener('scroll', handler);
}, []); // Пустой массив = монтирование и демонтирование
```

4. Инструменты мемоизации

Для предотвращения каскадных ререндеров тяжелых поддеревьев используются инструменты мемоизации:

- `useMemo` — кеширует результат ресурсоемких математических вычислений.
- `useCallback` — сохраняет неизменную ссылку на функцию между рендерами.
- `React.memo` — компонент высшего порядка, предотвращающий перерисовку дочернего компонента, если его пропсы не изменились поверхностно (shallow compare).

5. Ссылки и глобальный контекст

`useRef` возвращает неизменяемый объект-контейнер `{ current: value }`, изменение которого не инициирует рендер. Применяется для хранения ссылок на реальные DOM-узлы или идентификаторы таймеров.

`useContext` реализует шину данных для сквозной передачи глобальных настроек (темы, локализации) без ручного прокидывания пропсов через промежуточные слои.

Современная продакшен-архитектура и гибридный рендеринг

1. Разделение архитектурных слоев состояния

В масштабируемых веб-приложениях состояние разделяется на независимые уровни:

- **Server Cache State:** Серверные данные (кеширование, повторные запросы, инвалидация через TanStack Query).
- **Client UI State:** Локальное состояние интерфейса (модальные окна, сайдбары через Zustand или Redux Toolkit).
- **URL State:** Состояние фильтров и пагинации, хранящееся непосредственно в поисковой строке браузера.

2. Разделение кода (Code Splitting)

Разбиение монолитного JS-бандла на чанки с помощью динамических импортов `React.lazy` и компонента `Suspense` снижает начальный объем загружаемого кода:

```
import { lazy, Suspense } from 'react';
const AnalyticsModule = lazy(() => import('./AnalyticsModule'));

function App() {
  return (
    <Suspense fallback={<div>Загрузка модуля аналитики...</div>}>
      <AnalyticsModule />
    </Suspense>
  );
}
```

3. SSR, SSG и гидратация

Для решения проблем SEO и ускорения отображения первого экрана (LCP) индустрия использует Server-Side Rendering (SSR) и Static Site Generation (SSG). Сервер генерирует готовый HTML-текст, браузер моментально его рисует, а затем выполняется процесс **гидратации (Hydration)** — клиентский React связывает виртуальное дерево компонентов с существующей серверной разметкой и подключает обработчики событий.

4. Серверные компоненты (React Server Components, RSC)

RSC разделяет компоненты на два типа:

- **Server Components (по умолчанию):** Исполняются исключительно на сервере, имеют прямой доступ к базам данных и микросервисам по протоколам TCP/gRPC, а их код никогда не передается в бандле клиенту.
- **Client Components (маркируются директивой `'use client'`):** Загружаются браузером и отвечают за пользовательскую интерактивность (хуки состояния, анимации, клики).